Statistical Analysis of Frequency Response Functions: A MATLAB Library Reference Manual

Vittorio Lippi

March 9, 2026

Abstract

This technical report documents a comprehensive MATLAB library designed for the statistical analysis of Frequency Response Functions (FRFs). The library relies on transforming complex frequency-domain data into time-domain Pseudo Impulse Responses (PIRs) to facilitate robust statistical evaluation. By leveraging non-parametric bootstrapping techniques and Permutational Multivariate Analysis of Variance (PERMANOVA), the toolset allows researchers to construct confidence and prediction bands, compare group differences, and evaluate individual samples against a baseline without assuming a strict normal distribution of the data. This manual provides a didactic explanation of each function, bridging the gap between advanced mathematical programming and practical engineering application.

The full source code, along with the datasets used in the example scripts, is open-source and freely available at `https://github.com/VittorioFreiburg/FRF-statistics`.

How to Cite This Manual

```
@techreport{lippi2026frfmanual,
  title       = {Statistical Analysis of Frequency Response Functions: A
    MATLAB Library Reference Manual},
  author      = {Lippi, Vittorio},
  year        = {2026},
  month       = {March},
  type        = {Technical Report},
  institution = {Calejo},
  doi         = {10.5281/zenodo.18888166},
  url         = {https://doi.org/10.5281/zenodo.18888166}
        note        = {GitHub Repository: \url{https://github.com/
    VittorioFreiburg/FRF-statistics}}
}
```

Contents

1 Introduction **3**

2 Installation and Quick Start **3**
2.1 Installation 3
2.2 Quick Start Example 3

3 Data Representation and Tensor Dimensions **4**
3.1 SISO: Single-Input Single-Output 4
3.2 MIMO: Multi-Input Multi-Output 5
3.2.1 1. The 3D Matrix Approach 5
3.2.2 2. The Cell Array Approach 6
3.3 Internal Transformation: The Supervector 6

4 Library Functions **6**
4.1 FRF_pseudoimpulse 6
4.2 FRF_Supervector 6
4.3 FRF_ConfidenceBand 7
4.4 FRF_PredictionBand 8
4.5 FRF_ConfidenceBandDifference 8
4.6 FRF_MinimalPredictionBand 8
4.7 FRF_pdf 9
4.8 FRF_permanova 9
4.9 FRF_Plotbands 10

5 Example Scripts **10**
5.1 SCRIPT_Example.m 10
5.2 SCRIPT_Example_MIMO.m 10
5.3 SCRIPT_MinBandExample.m 10
5.4 Industrial Process Control & MIMO Anomaly Detection 11

1 Introduction

Frequency Response Functions (FRFs) are essential in engineering and applied sciences for understanding how systems respond to vibrational, acoustic, or external inputs across different frequencies. However, performing statistics directly on complex numbers in the frequency domain is mathematically cumbersome and often non-intuitive.

This library [5] solves that problem by first converting FRFs into time-domain signals called Pseudo Impulse Responses (PIRs) [6]. Once the data is in the time domain, the library uses a statistical technique called **bootstrapping**. Instead of assuming the data follows a perfect bell curve (a normal distribution), bootstrapping estimates variability by repeatedly drawing random samples from your existing empirical data. This allows for the creation of highly accurate confidence bands, prediction bands, and statistical tests even for complex, multi-input multi-output (MIMO) systems [10]. To assist researchers in selecting the appropriate statistical tool for their specific experimental design, Table 1 provides a quick reference guide mapping common scientific questions to their corresponding functions. Note that the "inspiration" column in this table reports legacy papers that utilize datasets highly suitable for such tests; these specific studies inspired the conceptualization and implementation of the respective functions, but it is important to clarify that they were written before this library was available and were conducted without it. A big inspiration for the present library comes from [4]. The use of PIRs allows to apply the functional statistics described in [4] (confidence and prediction bands) on data that are originally provided as complex domain FRFs so this library can be seen as an adaptation of [4] to complex functions. An "holistic" study of FRFs through their fourier transform is here regarded as more substantial and less arbitrary than using a specifically defined scalar function or applying multivariate methods such as Hotelling's T2 on the separate components of the FRF.

2 Installation and Quick Start

2.1 Installation

This library relies entirely on standard MATLAB functions and requires no external toolboxes. To install it, simply download the repository from GitHub and add its unzipped folder to your MATLAB path:

```
% Add the downloaded folder to your MATLAB path
addpath('C:\Path\To\Your\Download\FRF_Statistics_Library');
savepath; % Optional: saves the path for future sessions
```

2.2 Quick Start Example

To see the library in action, let's generate some mock FRF data and compute a 95% confidence band. In a real scenario, this data would come from your experimental sensors (like force plates or accelerometers).

```
% 1. Setup mock frequency vector and parameters
sf = 100;                   % Sampling frequency (Hz)
sample_time = 1/sf;
% Sample time
```

```
phi = 0.5:0.5:10;         % Frequencies from 0.5 to 10 Hz
N_samples = 15;
% Number of subjects/trials

% 2. Generate mock complex FRF data (N_samples x Length of phi)
% (Adding random noise to simulate experimental variation)
mock_FRFs = complex(rand(N_samples, length(phi)) + 1, ...
                    rand(N_samples, length(phi)) - 0.5);
% 3. Calculate the 95% Confidence Band (alpha = 0.95, 1000 bootstraps)
alpha = 0.95;
B = 1000;
[avg, sigma, band, Cc] = FRF_ConfidenceBand(mock_FRFs, phi, ...
                                            sample_time, alpha, B);
% 4. Generate a time vector for plotting
[~, time] = FRF_Supervector(mock_FRFs(1,:), phi, sample_time);
% 5. Plot the results using the included visual helper
figure;
FRF_Plotbands(time, avg, band, [0 0.4 0.7]);
% Blueish color
title('95% Confidence Band for Average PIR');
xlabel('Time (s)');
ylabel('Amplitude');
```

This minimal script takes raw frequency-domain numbers, handles the automatic time-domain transformation, runs 1000 statistical permutations, and plots the boundaries of where the true average of your system lies.

3 Data Representation and Tensor Dimensions

To effectively utilize the statistical tools in this library, it is crucial to understand how Frequency Response Functions (FRFs) are structured within the MATLAB environment before being passed to the functions. The library handles data as complex-valued arrays, and the dimensionality depends on whether you are analyzing a Single-Input Single-Output (SISO) or Multi-Input Multi-Output (MIMO) system.

3.1 SISO: Single-Input Single-Output

For a standard SISO experiment, a single FRF is represented as a **row vector** of complex numbers. Each column element corresponds to a specific frequency point defined in the frequency vector ϕ. When analyzing a group of subjects or trials, these individual row vectors are stacked vertically to form a 2D matrix.

	F **Frequencies**
	Sample 1 (Complex Row Vector)
	Sample 2 (Complex Row Vector)
N **Samples**	⋮
	Sample N (Complex Row Vector)

The dimensions of the SISO input matrix are $N \times F$, where:

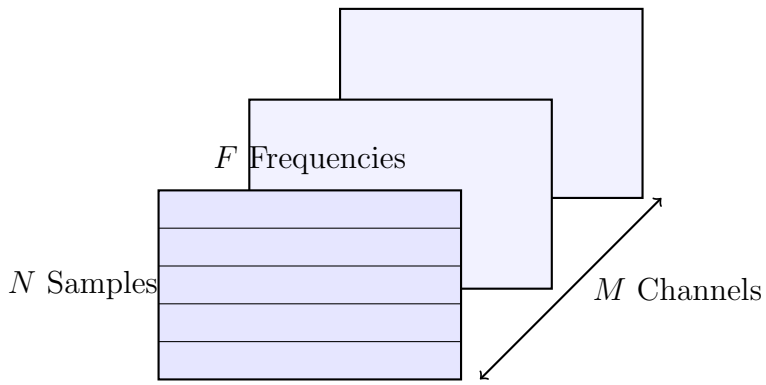
- N **(Number of Samples):** The number of independent observations, subjects, or trials.
- F **(Number of Frequencies):** The number of discrete frequency points evaluated (must match the length of ϕ).

3.2 MIMO: Multi-Input Multi-Output

In complex experiments involving multiple sensors or input-output relationships, the data structure expands to include a third dimension. The library's `FRF_Supervector` function is designed to ingest this multi-dimensional data. This can be represented in MATLAB in two ways:

3.2.1 1. The 3D Matrix Approach

If all input/output channels share the exact same frequency vector ϕ, a 3D array is used.



The dimensions for the 3D matrix are $N \times F \times M$, where:

- M **(Number of Channels):** The total number of unique input-output relationships being measured simultaneously.

3.2.2 2. The Cell Array Approach

For heterogeneous datasets where different channels might have different sampling parameters or frequency lengths, the library utilizes a **Cell Array**. The cell array has a size of $1 \times M$, where each individual cell contains an $N \times F$ matrix corresponding to a specific channel.

3.3 Internal Transformation: The Supervector

Regardless of whether you pass a 2D matrix, a 3D matrix, or a Cell Array, the library internally relies on the `FRF_Supervector` function. This function transforms each complex FRF into a time-domain Pseudo Impulse Response (PIR) and concatenates the channels horizontally. The resulting "Supervector" used for internal statistical calculations is always a flattened 2D matrix of dimensions $N \times L_{total}$, where L_{total} is the sum of the time-domain samples across all M concatenated channels.

4 Library Functions

4.1 FRF_pseudoimpulse

Purpose: Reconstructs a time-domain signal from complex FRF components.

```
[x, t] = FRF_pseudoimpulse(y, F, sf)
```

Parameters:

- `y`: Complex FRF values (the frequency data to be transformed).
- `F`: Vector of frequencies (in Hz) corresponding to the `y` values.
- `sf`: Sampling frequency (in Hz) for the desired time-domain output.
- `x`: The output time-domain signal (Pseudo Impulse Response).
- `t`: The generated time vector for plotting the signal.

Explanation: Think of this function as a translator. It takes how a system behaves at different continuous vibrations (frequencies) and translates it into how the system would "ring" if you struck it with a hammer once (time domain). The function does this by calculating a manual Inverse Fourier Transform. To ensure the time signal is accurate, it intelligently finds the "fundamental frequency" (the greatest common divisor of your input frequencies) to determine exactly how long the time signal needs to be [6].

4.2 FRF_Supervector

Purpose: Converts Multi-Input Multi-Output (MIMO) FRFs into concatenated raw PIRs.

```
[S, t] = FRF_Supervector(FRFs, phi, sample_time)
```

Parameters:

- `FRFs`: A 3D matrix (or cell array) of FRF sets.

- `phi`: The vector of frequencies.
- `sample_time`: The sample time for the resulting PIRs.
- `S`: The output "Supervector" matrix containing concatenated raw PIRs.

Explanation: When dealing with complex systems, you often have multiple sensors measuring data simultaneously (MIMO). Analyzing them one by one is tedious and misses the big picture. This function uses `FRF_pseudoimpulse` to convert all your sensors' frequency data into time data, and then stitches them side-by-side into one long, continuous sequence called a "Supervector" [10]. This allows the library's statistical engines to analyze the entire system's multidimensional behavior simultaneously as a single entity.

Handling DC Offsets (0 Hz): Time-domain transformations are highly sensitive to 0 Hz (DC) components, which can create artificial static offsets in the resulting Pseudo Impulse Responses (PIRs). This library deliberately leaves the frequency vector untouched to preserve your original data structure and assumptions. However, if you experience unwanted static offsets in your time-domain analysis, it is highly recommended that you manually filter out the 0 Hz component from both your frequency vector (`phi`) and your FRF data matrices prior to passing them into the library functions.

Motivation: Many experiments are testing the systems with multiple inputs and measuring multiple outputs, for example in posture control and balance [15, 16].

4.3 FRF_ConfidenceBand

Purpose: Calculates a confidence band for the average of a set of FRFs.

```
[avg, sigma, band, Cc, chist, values] =
    FRF_ConfidenceBand(FRFs, phi, sample_time, alpha, B)
```

Parameters:

- `alpha`: The desired statistical significance level (e.g., 0.95 for a 95% confidence band).
- `B`: The number of bootstrap repetitions (how many random samples to simulate).
- `band`: A two-row matrix containing the upper and lower boundaries of the confidence band.
- `Cc`: The threshold constant determined by the bootstrap process.

Explanation: A confidence band tells you where the *true average* of your system is likely to fall. Because we don't assume our data is perfectly uniform, this function uses a non-parametric "bootstrap" method [6]. Imagine pulling data points out of a hat, recording the average, putting them back, and repeating this `B` times (often 1000 or more). By looking at the maximum deviations across these thousands of "what-if" scenarios, the function calculates a precise threshold (`Cc`) to draw boundaries (`band`) around your average signal. **Comparing Paired Samples:** It is important to note that standard confidence bands can easily be used to compare groups of FRFs when your data consists of *paired samples* (for example, testing the exact same subject under two different experimental conditions). To do this, you do not need a specialized paired-test function. Instead, you first subtract the paired FRFs from each other to create a single

set of "difference FRFs" (Condition A - Condition B for each subject). Then, you simply compute a standard confidence band (`FRF_ConfidenceBand`) on this new difference set. To interpret the results, look at the horizontal axis corresponding to zero. If the zero line falls completely outside of your computed confidence band, you reject the null hypothesis. In plain terms: if the band representing the average difference does not touch zero, you have proven a statistically significant difference between the two conditions.

4.4 FRF_PredictionBand

Purpose: Calculates a prediction band for future, individual FRF observations.

```
[avg, sigma, band, Cp, chist, values] =
    FRF_PredictionBand(FRFs, phi, sample_time, alpha, B)
```

Explanation: While a *confidence* band shows where the average will be, a *prediction* band shows where the *next single measurement* is likely to land. Therefore, prediction bands are inherently wider. This function uses the same resampling logic as the confidence band but calculates a different threshold (`Cp`) that accounts for the natural spread and variation of individual data points, making it an excellent tool for quality control and spotting outliers [6]. `values`: The bin edges (threshold values) corresponding to the `chist` cumulative histogram. **Note:** It is a common misconception that this output contains the raw PIRs of every single bootstrap run. The raw PIRs are intentionally discarded during the loop to save memory. However, if you do not need to plot the empirical CDF, it is highly recommended to use the tilde (∼) operator (e.g., `[avg, sigma, band, Cp, ∼, ∼]`) to ignore these outputs, keeping your workspace clean and saving RAM.

4.5 FRF_ConfidenceBandDifference

Purpose: Computes confidence bands on the difference between the means of two distinct groups.

```
[avg, sigma, band, Cc, chist, values] =
    FRF_ConfidenceBandDifference(FRF1, FRF2, phi, sample_time, alpha, B, Bs)
```

Explanation: If you have two different batches of products (or a system before and after a modification), you want to know if their differences are mathematically significant or just random noise. This function ingests two sets of data (`FRF1` and `FRF2`). It uses a nested bootstrap approach to estimate standard deviations and evaluate the difference between the two group averages. If the resulting band does not cross zero, it proves a statistically significant difference exists between the two groups. The rigorous theoretical formulation for this unpaired test approach was introduced as supporting methodology for systems neuroscience research [12, 8].

4.6 FRF_MinimalPredictionBand

Purpose: Computes the minimum threshold needed to encompass a specific tested FRF.

```
[avg, sigma, band, Cp, chist, values, alpha] =
    FRF_MinimalPredictionBand(X, FRFs, phi, sample_time, B)
```

Explanation: Instead of asking "What is the 95% boundary?", this function asks the reverse: "I have a specific test sample (`X`); what boundary level (`alpha`) do I need to

draw so that it just barely fits inside?". It compares the test sample against the baseline `FRFs`. If the resulting `alpha` is extremely high (e.g., 0.999), it means your test sample is highly abnormal compared to the baseline, which is highly useful for damage detection or fault diagnosis [6, 7].

Note: I added this function (as well as FRF_pdf) to the library to provide a likehood estimation after a colleague at a conference asked me if the library provided a way to estimate the probability of a sample. I aknowledge that this test is a bit peculiar in that, instead of testing an hypothesis agaist data formulates and ad hoc hypothesis after the data is observed. In the meanwhile that we reflect about the epistemological drawbacks of this approach you can use it as a proxy of sample probability for your classification and diagnosis system.

4.7 FRF_pdf

Purpose: Estimates the Cumulative Density Function (CDF) and Probability Density Function (PDF) associated with a specific sample.

```
[cdf, pdf, sigma_cdf, sigma_pdf] =
    FRF_pdf(X, FRFs, phi, sample_time, B, metric)
```

Explanation: This function calculates the exact statistical probability of observing a specific sample (`X`) given a baseline set of data (`FRFs`). It allows you to specify a distance `metric`, either the sum of 'squared' residuals (emphasizing overall variance) or the 'max' difference (emphasizing peak deviations). It uses a smoothing window on the bootstrap results to plot smooth probability curves (PDF) and cumulative probabilities (CDF), helping users visualize exactly where a sample falls on the distribution curve of historical data [7].

Note: I added this function and FRF_MinimalPredictionBand presented above to provide a likehood estimation after being asked if the library provided a way to estimate the probability of a sample. Being based on euclidean distance between the FRFs (the different metrics you can use) the implementation of this function goes a bit agaist the aim of avoiding arbitrary scalar values to be used for the statistics, that was the main motivation behind developing this library on the basis of the Fourier transform of the FRFs (i.e. the PIRs). Anyways as it can be useful to estimate somehow the likehood of a sample, for example for anomaly detection, this function is kept here

4.8 FRF_permanova

Purpose: Performs Permutational Multivariate Analysis of Variance on FRF data.

```
[p_value, F_obs, R2, perm_F] =
    FRF_permanova(FRFs, groups, phi, sample_time, B)
```

Explanation: Standard MANOVA requires data to perfectly follow normal distributions. PERMANOVA bypasses this by using permutations (shuffling data randomly) [10]. After transforming the data into Supervectors, this function critically applies Z-Score Standardization. This ensures that signals with massive raw numbers don't unfairly overshadow signals with tiny raw numbers. It calculates how much variance exists *within* a group versus *between* groups. By shuffling the group labels `B` times, it generates a `p_value`: if the true grouping produces a variance score far higher than the shuffled random groupings, you can confidently state your groups are fundamentally different.

4.9 FRF_Plotbands

Purpose: A visual utility to plot time-domain averages and confidence/prediction bands.

```
h = FRF_Plotbands(time, avg, band, color)
```

Explanation: This is a straightforward graphical helper function. It takes the output from any of the band-generation functions and plots the average line solidly, alongside the upper and lower boundaries of the `band` as easily readable dash-dotted lines.

5 Example Scripts

To help users understand the practical implementation of the library, three comprehensive example scripts are included. These scripts demonstrate full analysis pipelines, from loading data to plotting statistical bands. The experimental datasets (provided as `.mat` files) used in these scripts are derived from real-world posture control and engineering studies, which are thoroughly described in their respective research papers. Table 2 provides an overview of these scripts.

5.1 SCRIPT_Example.m

Purpose: Standard group analysis and difference testing. **Key Functions Demonstrated:** `FRF_ConfidenceBand`, `FRF_ConfidenceBandDifference`, `FRF_Plotbands`.

Explanation: This script provides the foundational workflow for analyzing Single-Input Single-Output (SISO) frequency response functions. It loads two distinct datasets: one representing healthy control subjects and another representing patients with a neurological condition like Progressive Supranuclear Palsy [12]. The script first computes and plots a 95% confidence band for the healthy control group to establish a baseline of normal behavior. It then uses the unpaired test function (`FRF_ConfidenceBandDifference`) to directly compare the healthy group against the patient group [8]. Finally, it plots the difference band; if this boundary does not cross the zero-line, the script visually confirms a statistically significant difference in postural control between the two populations.

5.2 SCRIPT_Example_MIMO.m

Purpose: Multivariate analysis and PERMANOVA testing.

Key Functions Demonstrated: `FRF_ConfidenceBand`, `FRF_PredictionBand`, `FRF_ConfidenceBandDifference`, `FRF_permanova`. **Explanation:** Modern experimental setups often record multiple inputs and outputs simultaneously, creating complex matrices of data. This script demonstrates how the library handles these multi-dimensional MIMO arrays effortlessly via the "Supervector" under-the-hood concatenation [10]. Using mock multi-dimensional datasets, the script calculates both confidence and prediction bands for a combined multivariate system. Crucially, it demonstrates how to format grouping labels and execute `FRF_permanova` to obtain a formal p-value indicating whether the variance *between* the two MIMO datasets is statistically significant compared to the random noise *within* them.

5.3 SCRIPT_MinBandExample.m

Purpose: Evaluating an individual sample against a baseline population.

Key Functions Demonstrated: `FRF_MinimalPredictionBand`, `FRF_pdf`. **Explanation:** Instead of comparing groups, this script addresses the common scenario of evaluating a newly acquired single measurement. It loads a baseline dataset of ten normal subjects (`S10.mat`) and a single test sample (`X.mat`) [7]. The script first calculates the exact probability of observing sample `X` given the baseline data by estimating its Cumulative Density Function (CDF) and Probability Density Function (PDF) using `FRF_pdf`. Following this, it utilizes `FRF_MinimalPredictionBand` to calculate the tightest possible boundary that just barely includes the test sample. By outputting the threshold limit (`Cp`) and the empirical alpha level, the script provides a direct, quantifiable metric of how "abnormal" or "normal" the individual sample is compared to the historical baseline.

5.4 Industrial Process Control & MIMO Anomaly Detection

While much of the inspiration for this library stems from clinical neurology and posture control [12], the multi-dimensional nature of the `FRF_Supervector` makes it perfectly suited for industrial engineering and process control [10].

For instance, if you are monitoring a heavy machinery plant with multiple vibration sensors (inputs) and acoustic monitors (outputs), you can structure your 2x2 MIMO data as a 3D matrix.

Guidance on Tuning α (Confidence Levels): In clinical and formal academic settings, it is standard practice to set $\alpha = 0.95$ (a 95% boundary). However, in industrial applications such as predictive maintenance or early-warning plant alarms, you may purposefully want to tune this parameter down to $\alpha = 0.90$ or even 0.85. A lower α creates a tighter band, increasing the system's sensitivity to anomalies at the cost of a higher false-alarm rate. Tune this parameter based on your specific operational tolerance for false positives.

```
% Example: 20 historical machine runs, evaluated at 500 frequencies,
% across 4 channels (e.g., 2 inputs x 2 outputs flattened to M=4)
% mock_MIMO_data dimensions: [20 x 500 x 4]

% Calculate a multidimensional prediction band for the entire system
alpha = 0.90; % 90% confidence for early warning
B = 1000;
[avg, sigma, band] = FRF_PredictionBand(mock_MIMO_data, phi, sample_time,
    alpha, B);
```

Extracting Out-of-Bounds Points: Once you have established your statistical bands, you can dynamically find and highlight specific violations in a new test sample (`pir_test`). Because MATLAB requires identical matrix dimensions for logical operations, ensure both your test signal and your boundaries are formatted as column vectors:

```
% 1. Extract the upper and lower boundaries (utilizing the standardized row
    order)
band_lower = band(1, :)';
band_upper = band(2, :)';

% 2. Ensure the test sample is a column vector
pir_test = pir_test(:);

% 3. Find logical indices of all points violating the boundaries
```

```
out_of_bounds = (pir_test > band_upper) | (pir_test < band_lower);

% 4. Highlight violations on an existing plot
plot(time(out_of_bounds), pir_test(out_of_bounds), 'r*', 'MarkerSize', 8);
```

References

[1] Mustafa Emre Akçay, Vittorio Lippi, and Thomas Mergner. Visual modulation of human responses to support surface translation. *Frontiers in Human Neuroscience*, Volume 15 - 2021, 2021.

[2] Lorenz Assländer, Georg Hettich, and Thomas Mergner. Visual contribution to human standing balance during support surface tilts. *Human movement science*, 41:147–164, 2015.

[3] Lorenz Assländer and Robert J Peterka. Sensory reweighting dynamics in human postural control. *Journal of neurophysiology*, 111(9):1852–1864, 2014.

[4] Mark W Lenhoff, Thomas J Santner, James C Otis, Margaret GE Peterson, Brian J Williams, and Sherry I Backus. Bootstrap prediction and confidence bands: a superior statistical method for analysis of gait data. *Gait & Posture*, 9(1):10–17, 1999.

[5] Vittorio Lippi. Frf statistics library, 2024. originally available at: https://github.com/mcufidim/FRF-statistics.

[6] Vittorio Lippi. Bootstrap prediction and confidence bands for frequency response functions in posturography. *Experimental Techniques*, 2025.

[7] Vittorio Lippi. Statistical Tools for Frequency Response Functions from Posture Control Experiments: Estimation of Probability of a Sample and Comparison Between Groups of Unpaired Samples. In Golubitsky, Martin and Boccaletti, Stefano and Pinto, Carla M.A., editor, *Mathematical Approaches to Challenges in Biology and Biomedicine*, pages 285–306, Cham, 2025. Springer Nature Switzerland.

[8] Vittorio Lippi. Unpaired test for the comparison of frequency response functions groups. *arXiv preprint arXiv:2505.23778*, 2025.

[9] Vittorio Lippi, Lorenz Assländer, E Akcay, and Thomas Mergner. Body sway responses to pseudorandom support surface translations of vestibular loss subjects resemble those of vestibular able subjects. *Neuroscience Letters*, 736:135271, 2020.

[10] Vittorio Lippi, Leonard Johard, and Gabriele Landucci. Mimo and multi-group comparison problem in process control: A multivariate statistical framework for systems represented with frequency response functions. *ICINCO*, 2026. Under Review.

[11] Vittorio Lippi, Christoph Maurer, Christian Haverkamp, and Stefan Kammermeier. Modeling nonlinear head sway response induced by support surface tilt in healthy subjects. In *International Conference on Informatics in Control, Automation and Robotics*, pages 3–21. Springer, 2023.

[12] Vittorio Lippi, Christoph Maurer, Christian Haverkamp, and Stefan Kammermeier. Head posture control under perturbed conditions in progressive supranuclear palsy patients. *Frontiers in Systems Neuroscience*, Volume 19 - 2025, 2025.

[13] Vittorio Lippi, Christoph Maurer, and Thomas Mergner. Human-likeness indicator for robot posture control and balance. In *International Conference on Robotics, Computer Vision and Intelligent Systems*, pages 98–113. Springer, 2020.

[14] Vittorio Lippi, Christoph Maurer, and Thomas Mergner. Evaluating robot posture control and balance by comparison to human subjects using human likeness measures. In *Proceedings of the 2nd International Conference on Robotics, Computer Vision and Intelligent Systems (ROBOVIS 2021)*, volume 1, pages 77–85, Online, 2021. INSTICC, SciTePress.

[15] Vittorio Lippi, Christoph Maurer, and Thomas Mergner. Human body-sway steady-state responses to small amplitude tilts and translations of the support surface–Effects of superposition of the two stimuli. *Gait & Posture*, 100:139–148, 2023.

[16] Andrew R. Wagner, Sophia G. Chirumbole, Jaclyn B. Caccese, Ajit M. W. Chaudhari, and Daniel M. Merfeld. Development and validation of a two-dimensional pseudorandom balance perturbation test. *Frontiers in Human Neuroscience*, 18, 2024.

Question	**Test**	**Source**	**Inspiration**
What is the expected average behavior of this system, and how much uncertainty surrounds it?	Bootstrap Confidence Band (`FRF_ConfidenceBand`)	[6]	The case in which a subject is tested under different conditions is widely represented in literature, e.g.: eyes open vs eyes closed [3, 2, 1]
What is the expected range of values for a single future measurement of this system?	Bootstrap Prediction Band (`FRF_PredictionBand`)	[6]	Evaluating "human likenes" of humanoid behavior [13, 14], or in general making a diagnosis
Are these two independent groups of FRFs significantly different from each other?	Unpaired Bootstrap Test for Group Differences (`FRF_ConfidenceBandDifference`)	[12, 8]	Typically patients vs control [9]
How abnormal is this specific tested sample compared to the baseline population?	Minimal Encompassing Prediction Band (`FRF_MinimalPredictionBand`)	[6, 7]	[Placeholder citation]
Does the variance between multiple groups of MIMO FRFs exceed the variance within the groups?	Permutational Multivariate Analysis of Variance (`FRF_permanova`)	[10]	In [11], for example, two clinical conditions were compared with the healthy control group, a multiple group comparison could have been useful
What is the exact probability of observing this specific FRF sample given historical data?	Empirical Probability Density Function Estimation (`FRF_pdf`)	[7]	[Placeholder citation]

Table 1: Overview of scientific questions and their corresponding statistical tests within the library. The "source" column references the methodology's formal introduction. The "inspiration" column reports legacy papers featuring datasets that inspired the creation of these functions; these papers were written before this library was available and without its use.

Script Name	**Purpose & Workflow**	**Associated Literature**
`SCRIPT_Example.m`	Demonstrates standard confidence bands and tests for significant differences between two independent subject groups.	[12, 8]
`SCRIPT_Example_MIMO.m`	Expands the workflow to multidimensional Multi-Input Multi-Output (MIMO) arrays and executes a PERMANOVA test.	[10]
`SCRIPT_MinBandExample.m`	Focuses on evaluating a single test sample against a baseline, calculating encompassing bands and exact probabilities.	[7]

Table 2: Overview of the example scripts provided in the library, mapping their workflows to the corresponding published datasets and methodologies.